Draw It or Lose It

CS 230 Project Software Design Template

Version 1.2

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 3
Domain Model 4
Evaluation 4
Recommendations 8

Document Revision History

Version	Date	Author	Comments
1.0	01/30/2026	Isis Jackson	Initial
1.1	2/15/2026	Isis Jackson	Updated Evaluation
1.2	2/22/2026	Isis Jackson	Updated Recommendations

Executive Summary

The Gaming Room intends to develop a web-based game that will be compatible across multiple platforms, expanding upon their existing Android application, Draw It or Lose It. Unlike the current version, this new application will utilize an extensive library of stock drawings to provide visual clues, rather than requiring players to create images for team members to solve the puzzle.

Requirements

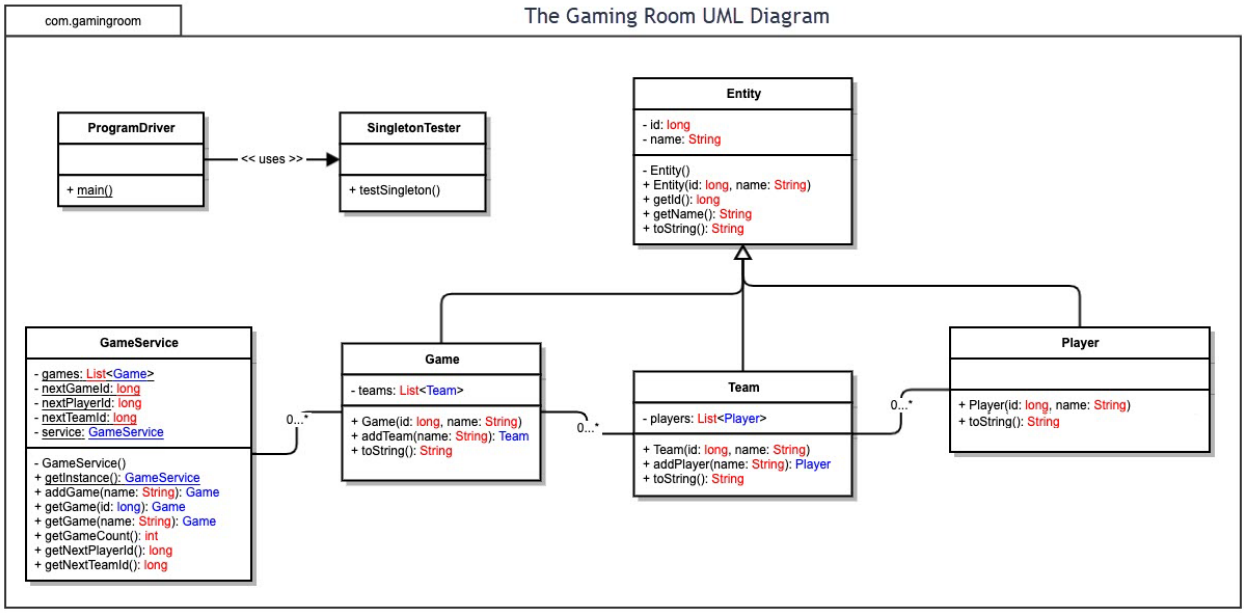
- The app must allow multiple teams to compete at the same time.
- Teams should have several players each.
- Game and team names must be unique to avoid conflicts.
- Only one game can run in memory at a time.
- The game should work on current web browsers for both desktop and mobile.

Design Constraints

- Distributed systems need fast communication to keep all players' views aligned in real time.
- The game is available as an Android app. We need to create a similar interface or consider updating the design with a new cohesive look.
- Handling unique identifiers for every game, team, and player requires a robust database to prevent race conditions when multiple users attempt to claim the same name.
- The game should support several teams, each with multiple players. This requires a client-server setup that allows the server to manage multiple players simultaneously.

System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.



Domain Model

The UML diagram (Unified Modeling Language) offers a visual overview of the game's system design. It illustrates how development will proceed, showing that the entity class links together the game, team, and player classes. Arrows on the diagram indicate inheritance from a superclass, while the layout displays relevant classes, variables, and methods for building the program. In the top left, the programDriver class points to SingletonTester, demonstrating that the programDriver relies on SingletonTester to check for a single instance restriction within the game memory. The GameService class contains complex methods that form the framework and functionality of the game. Because the program must create distinct games, teams, and players, each is represented as an individual class in the diagram. Lines between classes show their associations, with numbers indicating how many connections exist; for example, GameService may be linked to zero or more Game instances. This diagram will guide the development process and may be updated as needed.

Evaluation

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	Mac systems offer a high-quality, integrated experience since Apple designs both the hardware and software. They are known for their reliability, stability, and user-friendly design. A major advantage is	Linux is a highly flexible, community-driven operating system that offers a customizable environment for developers. Its modular design allows users to install	Windows Server is a more powerful, streamlined version of the Windows used at home, designed to handle large workloads and support hundreds of users simultaneously. Its strengths include	Mobile devices such as smartphones and tablets running iOS or Android are designed for personal, portable use and constant connectivity. Their main advantage is portability and they consume very little power compared to

Development Requirements	Mac	Linux	Windows	Mobile Devices
	their strong built-in security and excellent performance with graphics-intensive applications, making them ideal for visually demanding games. However, Macs come with a higher price tag—often referred to as the "Apple Tax"—and their hardware is less flexible to upgrade. As a result, scaling up performance can be more challenging compared to other platforms.	only what they need, resulting in fast and efficient performance. A key advantage is its low cost, as most distributions are free, and its lightweight interface ensures system resources stay focused on running applications. Linux is also widely supported in web development, with many tools built for it first. However, its reliance on command-line operations can be challenging for beginners, and without a centralized support system, users often depend on online communities for assistance.	familiarity—teams experienced with Windows can manage it easily—and seamless integration with Microsoft tools. It also benefits from strong commercial support backed by Microsoft. However, Windows Server requires significant system resources just to operate, which can limit efficiency. Additionally, its popularity makes it a target for cyberattacks, resulting in a need for ongoing updates and maintenance.	traditional servers. They can also be useful for quick demonstrations to show that an application is functioning. However, mobile hardware lacks the processing power needed to support large numbers of users, making it unsuitable for hosting high-demand applications. Connections can also be unreliable, as devices frequently switch between Wi-Fi and cellular networks. Additionally, mobile devices are not built for continuous, long-term operation and may overheat under sustained workload.
Client Side	Developing web-based games for macOS requires careful attention to hardware compatibility and specialized technical expertise. Projects typically takes 4–9 months and involves higher upfront costs, with Apple hardware and software licenses making development expensive, between	Supporting Linux clients requires focusing on open-source compatibility and a variety of hardware. Development is generally cost-effective, between 35,000 to $100,000, with additional savings though free tools. Projects usually take five to ten months. To ensure stable	For Window clients, performance optimization is the focus. Developing a mid-scale web-based game typically costs between $40,000 to $150,000, with annual maintenance at 15% of the original budget. Projects usually take four to eight months. Development teams should be proficient with DirectX or WebGL for hardware	Developing web-based mobile games requires working within the strict performance and power limitations of handheld devices. Costs typically range from $60,000 to $250,000, with annual maintenance around 20% of the original budget. Projects usually take six to twelve months, with much of the effort focused on performance optimization.

Development Requirements	Mac	Linux	Windows	Mobile Devices
	$50,000 to $200,000, plus maintenance and developer fees. To support consistent behavior across major browsers and on mobile platforms, developers must apply responsive web design techniques, adhere to W3C standards, use progressive enhancement with feature detection, map input events across device types, and conduct extensive cross-platform testing using services such as BrowserStack or LambdaTest.	performance, developers should be comfortable with the Linux command line, Bash scripting, and standard file-system practices. To maintain consistent performance across browsers and mobile devices, developers should emphasize standard design and responsive layouts. Following HTML5, CSS3, and ES6+ guidelines help reduce browser issues, while cross-platform engines support broad deployment. An input-translation layer ensures controls work smoothly on both desktop and mobile, and thorough testing verifies reliability across Linux systems and mobile operating systems.	acceleration and should understand Chromium-based optimization on Windows browsers. Developers should use flexible rendering engines which provide automatic fallback support for compatibility. Adaptive input handling allows the game to function well across desktop and mobile controls, while a standardized asset pipeline ensures consistent loading. Packaging the game as a Progressive Web App (PWA) enables installation and offline features, and automated cross-browser testing tools help verify reliability.	Development teams need strong expertise in touch events, gesture APIs, and Progressive Web App standards. To ensure seamless gameplay across mobile devices and desktop browsers, developers must manage viewports and aspect ratios through adaptive HTML and CSS so the interface fits a wide range of screen sizes. A unified input system allows the same logic to manage both touch and mouse events. Service workers enable offline functionality and remote debugging tools help identify and resolve device-specific issues, ensuring stable performance across diverse hardware.
Development Tools	Mac development depends on native languages like Swift and Objective-C, and tools such as Xcode and Visual Studio Code. Homebrew helps install utilities, and Git supports version control. Mac development requires Apple hardware, which increases upfront costs	Developing software for Linux means working within a flexible, open-source ecosystem that supports major programming languages and offers a wide range of free and professional tools. Essential development utilities provide strong	Developing software for Windows on the Microsoft system provides strong integration for building and deploying applications. Developers primarily use C#, C++, and Python. Windows teams benefit from integration with	Developing for mobile requires choosing between native builds and cross-platform frameworks that share a single codebase. Native iOS apps are written in Swift with Xcode on a Mac, while native Android apps use Kotlin with Android Studio. Cross-platform options such as Flutter and React

Development Requirements	Mac	Linux	Windows	Mobile Devices
	and often necessitates specialized teams for different target platforms. Cross-platform tools can help streamline staffing needs, and many developers value macOS for its stable Unix-based environment, which supports a productive and consistent workflow. Although many development tools are available for free, Mac developers face recurring expenses such as a $99/year Apple Developer Program fee for app distribution and notarization, and $299/year for the Enterprise Program for internal company distribution.	support. Linux also aligns closely with most production server environments, helping teams avoid platform-specific issues. Cross-platform frameworks allow developers to target multiple operating systems without maintaining separate native teams. Many Linux tools and distributions are free; some enterprise solutions do have costs. Red Hat Enterprise Linux requires paid support subscriptions, around $179/year for workstations and $799/year for servers. Premium development environments cost roughly $299/year, and frameworks like Qt provide free tiers but charge for commercial licenses.	services like Azure, Microsoft Entra, and GitHub Copilot, though this reliance can contribute to vendor lock-in. Depending on project needs, organizations may divide work between backend and frontend specialists or adopt cross-platform frameworks to support multiple operating systems. While Visual Studio provides a free edition for smaller teams, professional development often requires paid licensing. Visual Studio Professional is available as a monthly subscription or a license, while Visual Studio Enterprise offers advanced features at a higher cost.	Native enable instant delivery to both platforms. For testing and distribution, teams rely on TestFlight and Firebase App Distribution. For staffing, native development often requires separate iOS and Android teams, which can lead to different release dates. Publishing requires store fees. Apple charges $99/year; Google Play Console is a $25 one-time fee. Core tools like Xcode, Android Studio, and Visual Studio Code are free. Cloud services for notifications, crash reporting, and analytics typically offer free tiers that scale in cost with usage.

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform**
   Windows OS is the recommended operating platform. It is chosen for its widespread adoption, comprehensive software availability, and a broad range of Integrated Development Environments that facilitate rapid cross-platform development. This platform allows the game to transition from its Android-exclusive roots into a more accessible, web-based, or multi-platform environment.

2. **Operating Systems Architectures**
   The Windows architecture utilizes a Client-Server model integrated with web technologies to ensure scalability. It operates using a dual-mode system, Kernel Mode, which has direct hardware access for performance, and User Mode, where the game application runs to prevent system-wide crashes. This separation ensures a stable environment for high-performance gaming.

3. **Storage Management**
   A Relational Database Management System combined with Cloud-based storage is recommended. This configuration allows for the structured management of user profiles, game statistics, and team data while providing scalable storage for game assets that can be accessed globally.

4. **Memory Management**
   The Windows platform employs Virtual Memory and Paging techniques to manage resources. Specifically, Demand Paging is used to load only the necessary parts of the Draw It or Lose It software into physical RAM, while Disk Paging creates a larger virtual address space on the hard drive to prevent memory-related crashes during intense gameplay sessions.

5. **Distributed Systems and Networks**
   To enable communication between platforms, the game should utilize a distributed architecture where backend services are broken down into manageable components. These services communicate via RESTful APIs or WebSockets over both wired and wireless networks. To handle dependencies such as network outages, fault tolerance and redundancy must be implemented to ensure high availability.

6. **Security**
   Protection of user information is achieved through end-to-end encryption and firewalls provided by the Windows platform. Security measures include implementing TLS/SSL protocols for data in transit between devices and the server, alongside authentication and authorization frameworks to ensure only registered players can access specific team or game data. The platform's automated update system further protects against new vulnerabilities by ensuring the latest security patches are always active.